

NanoDB: Design and Implementation of a Custom Relational Database Engine with Constraint-Driven Data Structures

Qamar Mehmood

Roll Number: 26I-7800

CS-4002 Applied Programming · MS-CS Spring 2026

National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad

Instructor: Bushra Fatima

GitHub: <https://github.com/Qamar2001/nanodb>

Abstract

Building a database from the ground up is one of the most effective ways to understand how data systems actually work beneath the abstractions. This paper documents the design, implementation, and empirical evaluation of NANODB, a functional mini-relational database engine written in C++ without using any Standard Template Library (STL) containers. Every component was built by hand: the buffer pool, the LRU cache, the hash map, the AVL index, the SQL expression parser, and the join optimizer. The engine operates on 100,000 real records from the TPC-H benchmark [1], evaluates complex WHERE expressions through a Shunting-Yard parser on a custom stack, and routes multi-table joins using a graph-based MST algorithm. In live testing, the AVL tree index was roughly 69 times faster than a sequential scan on 20,000 records, and the buffer pool handled 4,950 LRU evictions correctly under a 50-page memory constraint. This paper covers the reasoning behind each design choice, the mathematical proof of each component's efficiency, and the measured results from the actual running system.

Introduction

There is a meaningful difference between knowing how to use a database and understanding how one actually works. A developer who writes SQL queries daily may never consider what happens inside the engine when that query arrives as a raw string, how the WHERE clause gets decomposed into executable operations, or why joining tables in a different order can change query time from milliseconds to minutes. The NANODB project was built to close that gap.

The project has one defining rule: no STL. There is no `std::vector`, no `std::map`, no

`std::stack`, no `std::sort`. Every data structure has to be written from scratch with raw arrays and pointers. This constraint is uncomfortable at first but turns out to be a surprisingly effective teaching tool. When you implement your own doubly-linked list and watch it handle thousands of cache evictions in microseconds while an equivalent array implementation would require millions of element shifts, the performance argument stops being abstract and becomes something you have experienced directly.

The resulting system, NANODB, is a working relational engine. It reads records from TPC-H benchmark files [1], stores them in a paged binary format on disk, indexes primary keys using a self-balancing AVL tree [2], evaluates SQL WHERE clauses using the Shunting-Yard algorithm on a custom stack, routes multi-table joins through a graph-based cost optimizer, and handles high-priority administrative queries through a binary max-heap scheduler. This paper documents how each of those pieces was designed, proves their complexity mathematically, and shows the empirical results from running them.

Related Work

Buffer Pool and Memory Management

Managing a fixed pool of memory pages and evicting the least recently used page to disk when the pool fills up is a foundational technique in database engineering. The CMU research group’s 2022 CIDR paper by Crotty et al. [3] provides a valuable modern perspective on this, arguing against the common shortcut of using OS memory-mapped I/O and demonstrating that an explicitly managed buffer pool gives the application far more control over what happens when memory runs short. Their findings reinforce the approach taken in NANODB, where every page eviction is handled deliberately through a doubly-linked list and hash map rather than delegated to the OS.

Neumann and Freitag [4] showed in the Umbra system that a disk-based engine does not have to sacrifice performance. Their design, which achieves in-memory-level query speeds while still using a page-based storage layer, demonstrates that the buffer pool architecture is not fundamentally limiting. It just has to be implemented carefully.

Indexing Structures

The AVL tree, the indexing structure used in NANODB, has been covered extensively in algorithms literature [2]. The key property is a strict height invariant: for any node, the heights of its two subtrees can differ by at most one. This keeps the tree height at $O(\log N)$ and guarantees that lookups never degrade no matter what order data is inserted.

More recent work has explored whether classical tree-based indexes can be replaced en-

tirely by learned models. Ding et al. [5] introduced ALEX at SIGMOD 2020, an adaptive learned index that can outperform B-tree variants on read-heavy, relatively static datasets by fitting a piecewise linear model to the data distribution. This is a compelling direction, but it comes with significant complexity: the model has to be retrained or adjusted as data changes, and it only works well when the data distribution is predictable. For a project where correctness and explainability matter more than squeezing out the last few percent of performance, the AVL tree’s straightforward invariant and four well-defined rotation cases are a better fit.

Query Optimization

Cost-based query optimization has a long history, but the field has seen significant renewed interest in the past few years through the application of machine learning. Marcus et al. [6] introduced Bao at SIGMOD 2021, a system that uses Thompson sampling to choose between pre-defined query plan “hints” based on historical execution data. Bao does not replace the underlying optimizer entirely but rather nudges it away from systematically bad choices, achieving substantial improvements on benchmarks like JOB and TPC-H.

NANODB’s join optimizer is far simpler than Bao, but the underlying concern is the same: given multiple tables to join, avoid the worst-case orderings. The approach here models tables as graph nodes and join costs as edge weights, then applies Kruskal’s MST algorithm [2] to find the globally cheapest join path. For queries over two or three tables this works well. The more sophisticated machinery in systems like Bao would become relevant once the number of joined relations grows large enough that the search space cannot be enumerated cheaply.

Leis et al. [7] address a related problem at the scheduling level, showing how query scheduling decisions in analytical workloads affect end-to-end throughput. In NANODB, scheduling is handled through a binary max-heap priority queue that puts administrative transactions ahead of background reads, a simple but practically important priority mechanism.

System Architecture

NANODB is organized as a layered pipeline. The storage layer at the bottom handles raw binary I/O through the Pager, which manages fixed-size pages and delegates residency tracking to the LRU Cache. Above the storage layer, the Schema Engine provides a polymorphic type system through `DataType`, `IntType`, `FloatType`, and `StringType` classes, where virtual comparison operators allow the rest of the engine to evaluate expressions without any type-checking if-statements. The Hash Map Catalog maps table names to

their registered entries in $O(1)$ expected time using the djb2 hash function with separate chaining. The Query Parser tokenizes SQL-like input and converts WHERE clauses to postfix form using the Shunting-Yard algorithm. The Executor resolves tables, selects between indexed and sequential scan access paths, evaluates postfix expressions row by row, and runs the MST optimizer for multi-table queries. A Priority Queue scheduler sits at the outermost layer, ensuring administrative queries preempt background work.

Memory Management: Buffer Pool and LRU Cache

Design

A database cannot load an entire dataset into RAM at once. The buffer pool provides an abstraction: a fixed number of memory pages are kept resident, and pages not currently needed are written to disk. When a new page must be loaded and the pool is full, the one that has gone longest without being accessed is evicted. That is the LRU policy.

The critical observation, noted by Crotty et al. [3], is that this eviction logic needs to be implemented explicitly rather than relying on OS-level memory mapping. With explicit control, the database knows exactly when a page is evicted, can ensure dirty pages are flushed before they leave memory, and avoids the unpredictable stalls that come from OS swap behaviour.

Implementation

NANODB's LRU cache combines two structures. A doubly-linked list maintains usage order: the head holds the most recently accessed page, the tail holds the least recently accessed. When a page is accessed, it moves to the head in $O(1)$ time by relinking its predecessor and successor. When eviction is needed, the tail is removed in $O(1)$ time. A hash map gives direct pointer access to any list node by page ID, so the cache can check residency without traversing the list.

Complexity

Let C be the pool capacity and B the number of hash buckets.

Space: $O(C + B)$.

Lookup, promotion, eviction: all $O(1)$ expected. The djb2 function distributes keys well across buckets, keeping chain lengths short. List operations always touch a fixed number of pointers regardless of pool size. This matches the performance model demonstrated for modern disk-based systems in [4].

Indexing: AVL Tree

The Problem

A query like `WHERE c_custkey == 10000` answered by a sequential scan has to examine up to N rows in the worst case. With 20,000 customers that means 20,000 comparisons for a single record. An index avoids this by mapping key values directly to row positions.

Implementation

NANODDB builds an AVL tree over the integer primary key of each registered table. Each node stores a key, a row ID, a height field, and left and right child pointers. The tree enforces the height invariant on every insertion through one of four rotation cases: LL, RR, LR, and RL. Each rotation is a fixed sequence of pointer assignments taking $O(1)$ time [2].

Complexity

The AVL height bound follows from the recurrence $N_h = 1 + N_{h-1} + N_{h-2}$, where N_h is the minimum nodes in an AVL tree of height h . Since this grows at least as fast as Fibonacci, $h < 1.44 \log_2(N + 2)$ for a tree with N nodes [2].

Space: $\Theta(N)$.

Search and insertion: $O(\log N)$. The contrast with learned indexes like ALEX [5] is instructive: ALEX achieves similar or better lookup times on static data by fitting a linear model to the key distribution, but requires significantly more infrastructure to maintain correctness as data is inserted. For this project the AVL tree's deterministic guarantees are preferable.

Query Parsing and Expression Evaluation

Tokenization

The parser scans the WHERE clause character by character, recognizing column names, integer and float literals, quoted strings, arithmetic operators (+, -, *, /, %), comparison operators (==, !=, <, >, <=, >=), logical operators AND and OR, and parentheses.

Shunting-Yard Conversion

The Shunting-Yard algorithm converts the infix token stream to postfix form using an output queue and an operator stack. Operand tokens go directly to the output. Operator tokens are held on the stack until an operator of lower or equal precedence ar-

rives, at which point higher-precedence operators are popped to the output first. Right parentheses pop until the matching left parenthesis is consumed. The precedence levels assigned are: OR = 1, AND = 2, comparisons = 3, addition/subtraction = 4, multiplication/division/modulo = 5. The full algorithm and its correctness proof appear in [2].

Evaluation

At query time the postfix stream is evaluated using a custom `Stack<DataType*>`. Column references are resolved to the current row’s cells and cloned. Literals are allocated as fresh `DataType` objects. Binary operators pop two operands, compute the result through virtual dispatch, and push the result back. The final value on the stack is the row’s evaluation result.

Complexity

Tokenization, Shunting-Yard conversion, and postfix evaluation each run in $O(T)$ time for T tokens. Total parsing is linear in the expression length.

Query Optimizer and Scheduling

Join Ordering via MST

Joining tables in the wrong order can produce unnecessarily large intermediate results and slow queries significantly. Even without the sophisticated machinery of a system like Bao [6], a simple cost-proxy model helps. NANODB represents each table as a graph node and each possible join pair as an edge whose weight equals the product of the two tables’ row counts. Kruskal’s algorithm [2] finds the minimum spanning tree, which gives the join ordering that connects all tables with the smallest total estimated intermediate result. For two or three tables, the full graph has at most three edges and the MST is computed instantly.

Priority Queue Scheduling

Queries enter the system through a binary max-heap. Standard queries receive priority 0; ADMIN-prefixed queries receive priority 100. The heap guarantees that the highest-priority pending query is always dequeued first, regardless of submission order. This maps directly to the scheduling model studied by Leis et al. [7] for analytical workloads, where query interleaving decisions matter for tail latency. Here the mechanism is simple, but it ensures that time-sensitive administrative operations are never blocked behind a flood of ordinary reads.

Heap insert and extract-max: $O(\log N)$ where N is the queue depth.

Empirical Evaluation

Setup

The engine was compiled with `g++ -std=c++17 -O2` on Windows 11. Timing used the C++ high-resolution clock at nanosecond precision. TPC-H data [1] was loaded at the full project scale: 20,000 customers, 30,000 orders, and 50,000 line items.

Index vs. Sequential Scan

The benchmark built tables of 1,000, 10,000, and 100,000 rows and measured the time to retrieve the last-inserted key using both a sequential scan (worst case: the target is at the end) and the AVL index.

Table 1: Lookup time comparison between sequential scan and AVL index across different table sizes.

Records	Seq. Scan (ns)	AVL Index (ns)	Speedup
1,000	2,800	200	14×
10,000	44,200	200	221×
100,000	1,494,500	1,700	879×

The sequential scan time grows roughly linearly with record count, consistent with $O(N)$. The AVL index time grows only slightly as the tree height increases, consistent with $O(\log N)$. From 1,000 to 100,000 records (a 100× size increase), index lookup time goes from 200 ns to 1,700 ns (an 8.5× increase), which matches $\log_2(100000)/\log_2(1000) \approx 5.6$ well. The practical upshot: at 20,000 customer records, the live demo showed a 69.4-fold speedup (131,900 ns sequential vs. 1,900 ns indexed). The kind of read-heavy workload where Ding et al.’s ALEX [5] would show further improvement over AVL is worth noting, but for an insert-update-query engine at this scale the AVL result is already very strong.

LRU Cache Stress Test

The buffer pool was capped at 50 pages and subjected to sequential access over a larger number of unique pages. Every access past the initial 50 causes an eviction, so the expected count is $\max(0, M - 50)$ for M unique touches.

All 4,950 evictions in the demo scenario completed without any memory errors. Because each eviction runs in $O(1)$ time through the linked list, the cumulative overhead was negligible compared to disk write time. This validates the concern raised by Crotty et al. [3]: explicit buffer management gives the application precise control over eviction

Table 2: LRU eviction counts under a 50-page buffer pool constraint.

Unique Pages	Evictions	Rate
1,000	950	95.0%
5,000	4,950	99.0%
10,000	9,950	99.5%

behaviour, something that would be difficult or impossible to guarantee with OS-level mmap.

Demo Test Cases

A: Parser. The query `(c_acctbal > 5000 AND c_mktsegment == "BUILDING") OR c_nationkey == 15` was parsed into postfix form, with the token sequence logged, and evaluated correctly against the customer table.

B: Index Optimizer. Sequential and indexed lookup times were both printed to the console for a specific `c_custkey` in the full dataset, showing the speedup in real time.

C: Join Optimizer. A three-table join over customer, orders, and lineitem ran after the MST optimizer logged its chosen join path.

D: Memory Stress. A 50-page pool processed 5,000 distinct page accesses and printed exactly 4,950 evictions, matching $\max(0, 5000 - 50) = 4950$.

E: Priority Queue. Fifty SELECT queries were enqueued, then one ADMIN INSERT was added. The INSERT was dequeued and executed first.

F: Complex Expression. The query `((o_totalprice * 1.5) > 100000 AND (o_custkey % 2 == 0)) OR (o_orderstatus != "0")` exercised arithmetic, modulo, comparisons, AND, OR, and nested parentheses simultaneously.

G: Persistence. Five customer records were inserted and serialized to disk. After a full restart with `--reload`, all five records were recovered and AVL indexes were rebuilt correctly.

Conclusion

The NANODB engine ended up being a genuinely working relational database. It stores 100,000 TPC-H records, evaluates complex SQL expressions, routes joins through a cost optimizer, manages memory through a correct LRU eviction policy, and persists data safely across restarts. None of it relied on the STL.

The benchmarks confirmed what the theory predicted. Index lookups were 879 times faster than sequential scans at 100,000 records. Thousands of LRU evictions ran without

errors. The MST optimizer picked the right join order. But the more durable value of the project is harder to put in a table. Writing an LRU cache from scratch and watching the constant-time pointer relinking handle thousands of evictions while an equivalent array would require millions of shifts makes the design principle concrete in a way that no textbook explanation quite manages. The same is true for the AVL rotations, the postfix evaluator, and the priority queue scheduler. Building these things yourself once is worth reading about them many times.

Future directions include extending the index to support range queries with a balanced BST traversal, replacing the toy MST optimizer with a proper dynamic programming approach for larger join sets as studied in systems like Bao [6], and experimenting with whether learned index structures like ALEX [5] could replace the AVL tree for read-heavy table scans.

References

- [1] Transaction Processing Performance Council, “TPC-H benchmark specification, revision 3.0.1,” Transaction Processing Performance Council, Tech. Rep., 2023. [Online]. Available: <https://www.tpc.org/tpch/>
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th. The MIT Press, 2022, ISBN: 978-0-262-04630-5.
- [3] A. Crotty, V. Leis, and A. Pavlo, “Are you sure you want to use MMAP in your database management system?” In *Conference on Innovative Data Systems Research (CIDR 2022)*, 2022. [Online]. Available: <https://db.cs.cmu.edu/papers/2022/cidr2022-p13-crotty.pdf>
- [4] T. Neumann and M. J. Freitag, “Umbra: A disk-based system with in-memory performance,” in *Conference on Innovative Data Systems Research (CIDR 2020)*, 2020. [Online]. Available: <https://www.cidrdb.org/cidr2020/papers/p29-neumann-cidr20.pdf>
- [5] J. Ding et al., “ALEX: An updatable adaptive learned index,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, New York, NY, USA: ACM, 2020, pp. 969–984. DOI: 10.1145/3318464.3389711
- [6] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, “Bao: Making learned query optimization practical,” in *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*, New York, NY, USA: ACM, 2021, pp. 1275–1288. DOI: 10.1145/3448016.3452838
- [7] V. Leis, A. Alhomssi, T. Ziegler, Y. Loeck, and C. Dietrich, “Self-tuning query scheduling for analytical workloads,” *Proceedings of the VLDB Endowment*, vol. 16, no. 9, pp. 2098–2111, 2023. DOI: 10.14778/3598581.3598583